

Lecture 9 – Fault Localization

AAA705: Software Testing and Quality Assurance

Jihyeok Park



2026 Fall

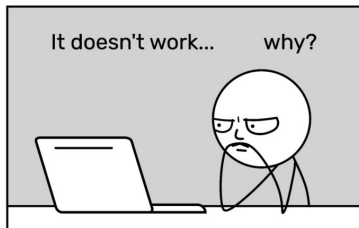
- Non-Testable Programs
 - No oracle, or an oracle as hard as the program itself
- Metamorphic Testing
 - A relation between two runs replaces the expected output – solvers, databases, compilers
- Differential Testing
 - Another implementation is the oracle – DNNs, Nezha, JIT on/off, N+1-version testing
- Property-based Testing
 - Properties plus generated inputs – QuickCheck, Hypothesis, **shrinking**

The Test Failed. Now What?

mid(2, 1, 3) should return 2. It returns 1.

```
1  int mid(int x, int y, int z) {  
2      int m = z;  
3      if (y < z) {  
4          if (x < y)      m = y;  
5          else if (x < z) m = y; // bug: m = x  
6      } else {  
7          if (x > y)      m = y;  
8          else if (x > z) m = x;  
9      }  
10     return m;  
11 }
```

- The failing test says the program is **wrong**. It does not say **which line**. Here there are 11; in a real project, thousands.
- **Today**: narrow it down. First shrink the **input**, then rank the **lines**.



- **Fault localization**: from a failing test to the **location** of the fault.
- By hand: print, assert, a breakpoint.
- Today: **automatically**, from the input, from the bug report, from the tests.

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
3. Spectrum-based Fault Localization (SBFL)
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize

1. Delta Debugging (DD)

- Recursive Delta Debugging – `ddmin`

- Delta Debugging on Changes

- Hierarchical Delta Debugging

- Probabilistic Delta Debugging (ProbDD)

- Delta Debugging for Program Debloating

2. Information Retrieval based Fault Localization (IRFL)

3. Spectrum-based Fault Localization (SBFL)

4. Mutation-based Fault Localization (MBFL)

5. Learning to Localize

Printing this page (896 lines of HTML) crashed Mozilla. Which part is the cause?¹

```
<td align=left valign=top>
<SELECT NAME="op sys" MULTIPLE SIZE=7>
<OPTION VALUE="All">All <OPTION VALUE="Windows 3.1">Windows 3.1
<OPTION VALUE="Windows 95">Windows 95 <OPTION VALUE="Windows 98">Windows 98
... (30 more operating systems) ...
<OPTION VALUE="other">other</SELECT></td> <td align=left valign=top>
<SELECT NAME="priority" MULTIPLE SIZE=7>
<OPTION VALUE="--">-- <OPTION VALUE="P1">P1 <OPTION VALUE="P2">P2
... (880 more lines) ...
```

57 runs, line by line: `<SELECT NAME="priority" MULTIPLE SIZE=7>`

26 more, char by char: `<SELECT>`

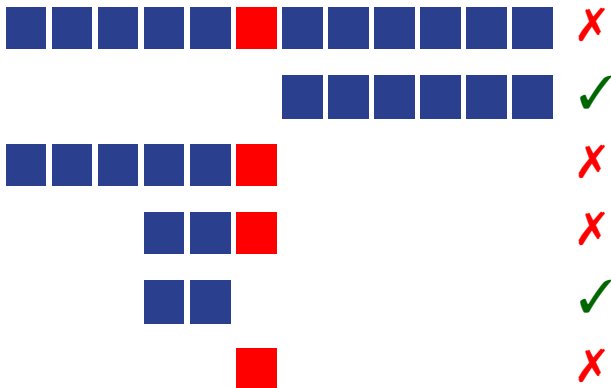
¹[[Mozilla bug 24735](#)] "*Printing causes mozilla to crash*" (2000). Numbers from [TSE'02].

Delete a part and run again. Still fails? Keep it deleted. Repeat.²

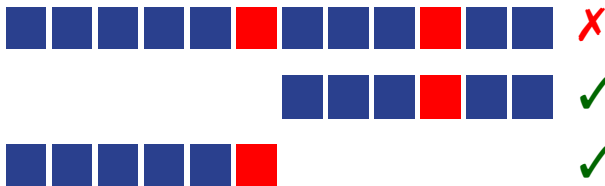
- The result: a **small** input that still fails.
- Easy to **read** and to **report**. Two reports that shrink to the same input are the **same bug**.
- **Shrinking** in property-based testing is the same idea.

²[TSE'02] A. Zeller, R. Hildebrandt. "Simplifying and Isolating Failure-Inducing Input."

The simplest strategy is **binary search**. Red is the part that matters:



But what if the failure needs **two** parts? Either half alone passes:



So try to **keep** a chunk, then to **remove** a chunk. If neither works, cut **smaller** chunks.

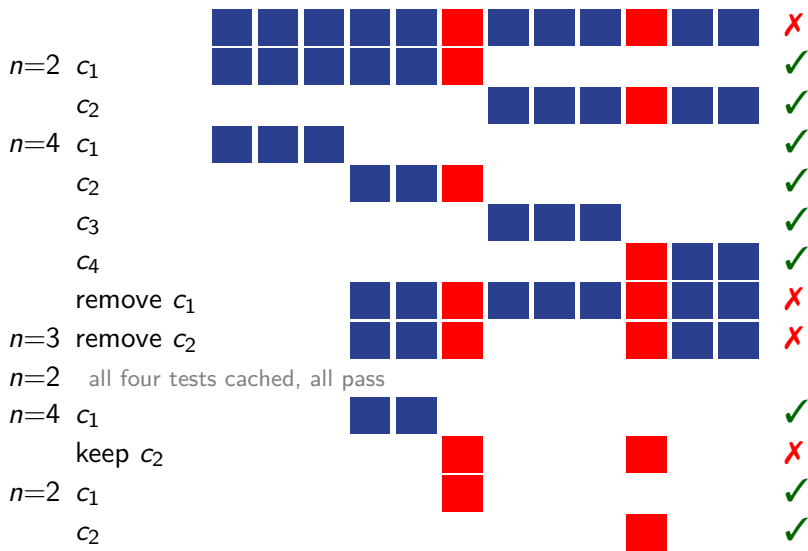
Algorithm ddmin

Input: a failing test case t **Output:** a **1-minimal** failing test case t'

```

1: function DDMIN( $t$ )
2:    $L \leftarrow$  the list of elements in  $t$ ;    $n \leftarrow 2$ 
3:   while  $n \leq |L|$  do
4:      $\langle c_1, \dots, c_n \rangle \leftarrow$  split  $L$  into  $n$  chunks
5:     if  $\exists i \in [1, n]. \text{test}(c_i) = \text{X}$  then
6:        $\langle L, n \rangle \leftarrow \langle c_i, 2 \rangle$                                  $\triangleright$  keep a chunk
7:     else if  $\exists i \in [1, n]. \text{test}(L \setminus c_i) = \text{X}$  then
8:        $\langle L, n \rangle \leftarrow \langle L \setminus c_i, \max(n - 1, 2) \rangle$          $\triangleright$  remove a chunk
9:     else if  $n < |L|$  then
10:       $n \leftarrow \min(2n, |L|)$                                  $\triangleright$  smaller chunks
11:     else
12:       break
13:   return the test case  $t'$  that corresponds to  $L$ 
  
```

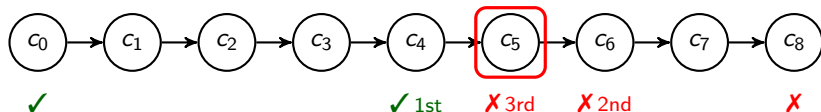
1-minimal: no **single** element can go. Not the smallest. $O(|t|^2)$ runs.



12 runs, with repeated tests cached. Every rule fired.

Which Commit Broke It?

The parts can also be the **commits** between a passing and a failing version.³



- One bad commit: **binary search**. c₅ in **3** runs out of 8. This is git bisect.
- At Google, a **flaky** test can lie once and blame the wrong commit. So each run counts as noisy evidence, with a probability per commit.⁴

³[ESEC/FSE'99] A. Zeller. "Yesterday, my Program Worked. Today, it Does Not. Why?"

⁴[ICST'23] T. A. D. Henderson, B. Dorward, E. Nickell, C. Johnston, A. Kondareddy. "Flake Aware Culprit Finding."

Delete random characters from a **C program** and you get a **syntax error**, not the bug.⁵

- **HDD** runs `ddmin` on the **parse tree**, one level at a time: functions, then statements, then expressions.
- Every try is a **valid** program.
- Compiler bugs are reported this way today (**C-Reduce**, **Perses**). New tools also ask an LLM for smaller rewrites.⁶

⁵[ICSE'06] G. Misherghi, Z. Su. "HDD: Hierarchical Delta Debugging."

⁶[ISSTA'24] M. Zhang, Y. Tian, Z. Xu, Y. Dong, S. H. Tan, C. Sun. "LPR: Large Language Models-Aided Program Reduction."

Level 1, **functions**: which of the three can go?

```
int copy(double to[],
         double from[], int count)
{
    int n= (count+7)/8;
    switch (count%8) do {
        case 0: *to++ = *from++;
        case 7: *to++ = *from++;
        case 6: *to++ = *from++;
        case 5: *to++ = *from++;
        case 4: *to++ = *from++;
        case 3: *to++ = *from++;
        case 2: *to++ = *from++;
        case 1: *to++ = *from++;
    } while (--n > 0);
    return (int)mult(to,2);
}
```

```
double mult( double z[], int n )
{
    int i;
    int j;
    for (j= 0; j< n; j++) {
        i= i+j+1;
        z[i]=z[i]*(z[0]+0);
    }
    return z[n];
}

int main( int argc, char *argv[] )
{
    double x[20], y[20];
    double *px= x;
    while (px < x + 20)
        *px++ = (px-x)*(20+1.0);
    return copy(y,x,20);
}
```

The final result. Every level removed only what the failure did not need:

```
double mult( double z[], int n )
{
    int i;
    int j;
    for (;;) {
        i= i+j+1;
        z[i]=z[i]*(z[0]+0);
    }
}
```

ddmin cuts **blindly**. **ProbDD** keeps, for each element, the chance that the failure **needs** it, and deletes the least needed ones.⁷

	s1	s2	s3	s4	s5	s6	s7	s8	
	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	
1	s1	s2	s3	s4	s5	s6	s7	s8	F
	0.3657	0.3657	0.3657	0.2500	0.2500	0.2500	0.2500	0.3657	
2	s1	s2	s3	s4	s5	s6	s7	s8	T
	0.3657	0.3657	0.3657	0	0	0	0	0.3657	
3	s1	s2	s3	s4	s5	s6	s7	s8	F
	0.6119	0.3657	0.3657	0	0	0	0	0.6119	
4	s1	s2	s3	s4	s5	s6	s7	s8	F
	0.6119	0.6119	0.6119	0	0	0	0	0.6119	
5	s1	s2	s3	s4	s5	s6	s7	s8	T
	0.6119	0	0.6119	0	0	0	0	0.6119	
6	s1	s2	s3	s4	s5	s6	s7	s8	F
	0.6119	0	1	0	0	0	0	0.6119	
7	s1	s2	s3	s4	s5	s6	s7	s8	T
	0	0	1	0	0	0	0	0.6119	
8	s1	s2	s3	s4	s5	s6	s7	s8	F
	0	0	1	0	0	0	0	1	

Two rows per run: test the **teal** elements, then update every chance (**orange**). Still fails (**T**): the deleted ones become **0**. Passes (**F**): they go **up**.

⁷[ESEC/FSE'21] G. Wang, R. Shen, J. Chen, Y. Xiong, L. Zhang. "Probabilistic Delta Debugging."

Look inside the math: each chance only goes **up**, one step per rejected deletion. It is just a **counter**.⁸

- **CDD**: one counter per element, delete the smallest first. Same results as ProbDD on 76 benchmarks.
- What really helped: **never repeat** a deletion already tried. The randomness did not matter.
- Lesson: when a technique works, ask **which part** works.

⁸[ICSE'25] M. Zhang, Z. Xu, Y. Tian, X. Cheng, C. Sun. "Toward a Better Understanding of Probabilistic Delta Debugging."

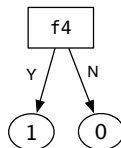
New test: not “*still crashes?*” but “*still does what I need?*” Then DD removes the rest.⁹ Seven functions, main calls only f1. Plain DD: **16** runs.

	f1	f2	f3	f4	f5	f6	f7	main	✓
1	f1	f2	f3	f4	f5	f6	f7	main	✗
2	f1	f2	f3	f4	f5	f6	f7	main	✗
3	f1	f2	f3	f4	f5	f6	f7	main	✗
4	f1	f2	f3	f4	f5	f6	f7	main	✗
5	f1	f2	f3	f4	f5	f6	f7	main	✗
6	f1	f2	f3	f4	f5	f6	f7	main	✗
7	f1	f2	f3	f4	f5	f6	f7	main	✗
8	f1	f2	f3	f4	f5	f6	f7	main	✓
9	f1	f2	f3	f4	f5	f6	f7	main	✓
10	f1	f2	f3	f4	f5	f6	f7	main	✗
11	f1	f2	f3	f4	f5	f6	f7	main	✗
12	f1	f2	f3	f4	f5	f6	f7	main	✗
13	f1	f2	f3	f4	f5	f6	f7	main	✗
14	f1	f2	f3	f4	f5	f6	f7	main	✗
15	f1	f2	f3	f4	f5	f6	f7	main	✓
16	f1	f2	f3	f4	f5	f6	f7	main	✓

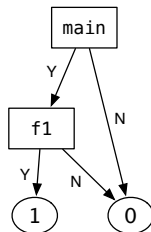
⁹[CCS'18] K. Heo, W. Lee, P. Pashakhanloo, M. Naik. “Effective Program Debloating via Reinforcement Learning.”

Chisel learns a decision tree from past runs to guess **which subset will pass**. Fewer runs, same result:

	f1	f2	f3	f4	f5	f6	f7	main	✓
1	f1	f2	f3	f4	f5	f6	f7	main	✗
2	f1	f2	f3	f4	f5	f6	f7	main	✗
3	f1	f2	f3	f4	f5	f6	f7	main	✗
4	f1	f2	f3	f4	f5	f6	f7	main	✓
5	f1	f2	f3	f4	f5	f6	f7	main	✓
6	f1	f2	f3	f4	f5	f6	f7	main	✗
7	f1	f2	f3	f4	f5	f6	f7	main	✓
8	f1	f2	f3	f4	f5	f6	f7	main	✓
9	f1	f2	f3	f4	f5	f6	f7	main	✗
10	f1	f2	f3	f4	f5	f6	f7	main	✗



After run 1



After run 6

A node asks whether a function is **kept**; a leaf guesses pass (**1**) or fail (**0**). One run in, it guesses `f4`. Six runs in, it knows: `main` and `f1`. Chisel tries first what the tree expects to pass.

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
 - Vector Space Model (VSM)
 - Tf-Idf
 - Cleansing Bug Reports and Source Code
3. Spectrum-based Fault Localization (SBFL)
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize

A bug report is **written in English** and names the things it is about. Can we use those words to find the file?¹⁰

 **thorstenr_42** (Reporter)
Description • 5 years ago

User Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:28.0) Gecko/20100101 Firefox/28.0 (Beta/Release)
Build ID: 20140410211200

Steps to reproduce:

1. create a folder in the bookmarks-toolbar and fill it with some bookmarks (one is enough)
2. opening the folder per mouse-leftclick works flawlessly but per touch-click it just opens for a fraction of a second and is immediately collapsed again

(Tested with installed ubuntu 14.04 and untouched live version on t440s with touchscreen)

Actual results:

Bookmarks-folders is immediateky closed again if opened per touchscreen

Expected results:

It should just open the folder and show the contents

¹⁰[\[Mozilla bug 1001903\]](#) "Can't open folder in Bookmarks toolbar with touchscreen: closes immediately."

*After **creating** a **folder** in the **bookmark toolbar** and a **bookmark** inside the **folder**, mouse left-click successfully opens the folder but **touch-click** fails to **open** it: it **opens** but immediately **closes** itself.*

You would open a file with **bookmark**, **folder**, and **touch** in it. **IRFL** does this: the report is the **query**, the files are the **documents**, and the answer is the most **similar** file.

Count the words. The report and each file become a **vector**:

	folder	bookmark	touch	open	close	print
report q	2	2	1	2	1	0
BookmarkFolder.js	5	4	0	3	2	0
TouchHandler.js	0	0	6	2	1	0
Printer.js	0	0	0	1	0	7

Similarity = the **angle** between two vectors, $\cos(d, q) = \frac{d \cdot q}{\|d\| \|q\|}$:

BookmarkFolder.js	TouchHandler.js	Printer.js
0.95	0.46	0.08

The ranking **is** the answer: read BookmarkFolder.js first.

open is in every file, so it tells us nothing. Weight each word by how often it is in the file (**tf**) and how **rare** it is among the N files (**idf**):

$$tfidf(t, d) = \underbrace{f_{t,d}}_{\text{term frequency}} \times \underbrace{\log \frac{N}{n_t}}_{\text{inverse document frequency}} \quad \begin{array}{l} f_{t,d}: \text{count of } t \text{ in } d \\ n_t: \text{files with } t \end{array}$$

	folder	bookmark	touch	open	close	print
$\log(3/n_t)$	1.10	1.10	1.10	0	0.41	1.10

New cosines:

	BookmarkFolder.js	TouchHandler.js	Printer.js
counts	0.95	0.46	0.08
tf-idf	0.94	0.34	0.00

Printer.js matched the report only on **open**. Now it scores 0.

Before counting, both sides are turned into the **same** kind of words:

Step	Example
Tokenize, lowercase	"Bookmarks toolbar" → bookmarks, toolbar
Stop words	"in the folder" → folder
Stemming	closes, closing → close
Split identifiers	getFileName → get, file, name
Keywords = stop words	return, if, while → dropped

The last two are what makes **code** searchable by **English**: a report says "file name", the code says `getFileName`.

Many reports already contain the answer:

```
Folder in bookmarks toolbar closes immediately on touch.  
TypeError: this._popup is null  
    at openFolder (BookmarkFolder.js:212)  
    at onTouchEnd (TouchHandler.js:88)
```

- IRFL only has to **copy** the file name. Such reports make IRFL look better than it is.¹¹
- **BugLocator** also uses the files fixed for **similar old reports**: the buggy file is in the top 10 (of 12,863) for 62.6% of Eclipse bugs.¹²

¹¹[ASE'14] P. S. Kochhar, Y. Tian, D. Lo. "Potential Biases in Bug Localization: Do They Matter?"

¹²[ICSE'12] J. Zhou, H. Zhang, D. Lo. "Where Should the Bugs Be Fixed? More Accurate Information Retrieval-Based Bug Localization Based on Bug Reports."

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
3. Spectrum-based Fault Localization (SBFL)
 - Genetic Programming for SBFL
 - Theoretical Analysis
 - Method-level Aggregation
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize

Back to mid. Six tests, one fails. Which tests **run** the faulty line?

```
1  int mid(int x, int y, int z) {  
2      int m = z;  
3      if (y < z) {  
4          if (x < y)      m = y;  
5          else if (x < z) m = y; // bug  
6      } else {  
7          if (x > y)      m = y;  
8          else if (x > z) m = x;  
9      }  
10     return m;  
11 }
```

	input	expected	
t_1	(3, 3, 5)	3	✓
t_2	(1, 2, 3)	2	✓
t_3	(3, 2, 1)	2	✓
t_4	(5, 5, 5)	5	✓
t_5	(5, 3, 4)	4	✓
t_6	(2, 1, 3)	2	✗

Idea: a line run by the **failing** test but not by the **passing** ones is suspicious.
Line 5 is run by t_6 – and by t_1 .

For each line, four numbers: covered by a **p**assing test (e_p), by a **f**ailing one (e_f), and **n**ot covered (n_p , n_f).

line	t_1	t_2	t_3	t_4	t_5	t_6	e_p	e_f	n_p	n_f
2 $m = z$	•	•	•	•	•	•	5	1	0	0
3 if ($y < z$)	•	•	•	•	•	•	5	1	0	0
4 if ($x < y$)	•	•			•	•	3	1	2	0
4 $m = y$		•					1	0	4	1
5 else if ($x < z$)	•				•	•	2	1	3	0
5 m = y	•					•	1	1	4	0
7 if ($x > y$)			•	•			2	0	3	1
7 $m = y$			•				1	0	4	1
8 else if ($x > z$)				•			1	0	4	1
8 $m = x$							0	0	5	1
10 return m	•	•	•	•	•	•	5	1	0	0

The **spectrum** is all a formula sees: coverage, plus pass/fail per test.

The first SBFL formula, **Tarantula**¹³:

$$Tarantula(e) = \frac{\frac{e_f}{e_f + n_f}}{\frac{e_f}{e_f + n_f} + \frac{e_p}{e_p + n_p}}$$

$$\underbrace{\frac{e_f}{e_f + n_f}}$$

fraction of **failing** tests that cover e

$$\underbrace{\frac{e_p}{e_p + n_p}}$$

fraction of **passing** tests that cover e

Suspicious = **run by failures, not by passes**. Every other formula is a variation of this.

¹³[ICSE'02] J. A. Jones, M. J. Harrold, J. Stasko. "Visualization of Test Information to Assist Fault Localization." [ASE'05] J. A. Jones, M. J. Harrold. "Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique."

One failing test, five passing. First fraction: 1 if t_6 runs the line. Second: $e_p/5$.

line	$\frac{e_f}{e_f+n_f}$	$\frac{e_p}{e_p+n_p}$	Tarantula	rank
5 m = y	1	0.2	0.83	1
5 else if (x < z)	1	0.4	0.71	2
4 if (x < y)	1	0.6	0.63	3
2, 3, 10	1	1.0	0.50	4
4 m = y, 7, 8	0	0-0.4	0	7

The fault ranks **first**. Why only 0.83? t_1 also runs it and still passes: **coincidental correctness**.

It began as a **visualization**: colour each line from red (only failures cover it) to green (only passes do).



Three Formulae, One Spectrum

Over 30 formulae exist. Two common ones – **Ochiai**, the usual default,¹⁴ and **Op2**.¹⁵

$$Ochiai = \frac{e_f}{\sqrt{(e_f + n_f)(e_f + e_p)}}$$

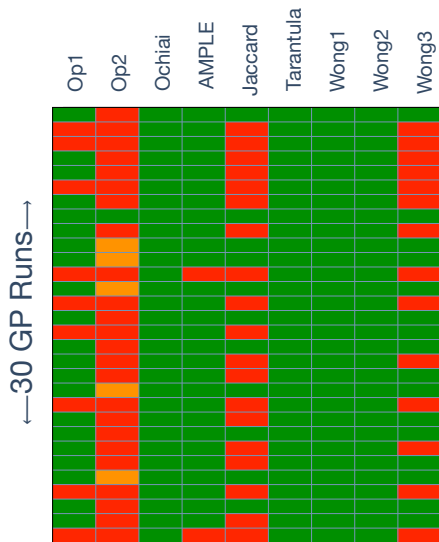
$$Op2 = e_f - \frac{e_p}{e_p + n_p + 1}$$

	5 m = y	5 else if	4 if	2, 3, 10
(e_f, e_p)	(1, 1)	(1, 2)	(1, 3)	(1, 5)
Tarantula	0.83	0.71	0.63	0.50
Ochiai	0.71	0.58	0.50	0.41
Op2	0.83	0.67	0.50	0.17

Different numbers, **same order**. With one failing test, each formula just sorts by e_p .

¹⁴[TAIC PART'07] R. Abreu, P. Zoetewij, A. J. C. van Gemund. "On the Accuracy of Spectrum-Based Fault Localization."

¹⁵[TOSEM'11] L. Naish, H. J. Lee, K. Ramamohanarao. "A Model for Spectra-Based Software Diagnosis."



4 Unix tools w/ 92 faults: 20 for training, 72 for evaluation.

- A formula is a tree over e_f, e_p, n_f, n_p . So **evolve** it. Fitness: the rank of the real fault.
- 30 GP runs vs. 9 human formulae. **Green**: GP wins, **orange**: tie, **red**: human wins.
- GP **always** beats Tarantula and Ochiai. It almost **never** beats Op2.

¹⁵[SSBSE'12] S. Yoo, "Evolving Human Competitive Spectra-Based Fault Localisation Techniques."

Two failing tests f_1, f_2 and four passing. Line A is run by f_1 only. Line B is run by both failures and two passes.

	(e_f, n_f, e_p, n_p)	Tarantula	Ochiai	Op2
A	$(1, 1, 0, 4)$	1.00	0.71	1.0
B	$(2, 0, 2, 2)$	0.67	0.71	1.6

- With **one** fault, every failing test runs it. f_2 does not run A , so A is **not** the fault. Tarantula still puts it first.
- Op2 always puts lines run by **all** failures on top. For one fault, no formula beats it.¹⁶ That is why GP could not beat it.

¹⁶[TOSEM'13] X. Xie, T. Y. Chen, F.-C. Kuo, B. Xu. "A Theoretical Analysis of the Risk Evaluation Formulas for Spectrum-Based Fault Localization."

```
public void testMe(int a) {  
    util();  
    if (a % 3 == 0) {  
        ... // faulty code  
    } else {  
        ...  
    }  
}  
public void util() {  
    ...  
}
```

- Every test that reaches the fault also runs `util()`. So both methods get the **same** score.
- Fix: rank **statements**, then give each method the score of its **best** statement. `testMe` wins.

Developers read **methods**, so rank methods.¹⁷

¹⁷[ISSTA'17] J. Sohn, S. Yoo. "FLUCCS: Using Code and Change Metrics to Improve Fault Localization."

```
int div(int a, int b) {  
    // missing: if (b == 0) ...  
    return a / b;  
}
```

Missing code: the faulty line does not **exist**, so no test runs it. SBFL points at **return** a / b, the symptom.

- **Many faults:** their failing tests mix in one spectrum, and each fault lowers the other's score.
- **Coincidental correctness:** like t_1 in mid, each test that runs the bug and passes pushes the fault **down**.

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
3. Spectrum-based Fault Localization (SBFL)
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize

A spectrum only says a line **ran**. MBFL **changes** the line (a **mutant**) and runs the tests again.¹⁸

change a ...	failing tests	passing tests
faulty line	some now pass (fixed)	few now fail
correct line	still fail	many now fail (broken)

So a line is **more suspicious** when changing it **fixes** failing tests, and **less** when changing it **breaks** passing ones.

¹⁸[ICST'14] S. Moon, Y. Kim, M. Kim, S. Yoo. "Ask the Mutants: Mutating Faulty Programs for Fault Localization."
[STVR'15] M. Papadakis, Y. Le Traon. "Metallaxis-FL: Mutation-Based Fault Localization."

MUSE: **add** the failing tests a mutant fixes, **subtract** the passing tests it breaks. Average over the mutants $mut(s)$ of s :¹⁹

$$\mu(s) = \frac{1}{|mut(s)|} \sum_{m \in mut(s)} \left(\underbrace{\frac{|f_P(s) \cap p_m|}{|f_P|}}_{\text{fail} \rightarrow \text{pass}} - \alpha \cdot \underbrace{\frac{|p_P(s) \cap f_m|}{|p_P|}}_{\text{pass} \rightarrow \text{fail}} \right)$$

- f_P, p_P : **all** failing / passing tests on the program P . $f_P(s), p_P(s)$: **those** that run s . p_m, f_m : tests that pass / fail on mutant m .
- α : $\text{pass} \rightarrow \text{fail}$ is much more common, so it gets a smaller weight.

¹⁹[ICST'14] S. Moon, Y. Kim, M. Kim, S. Yoo. "Ask the Mutants: Mutating Faulty Programs for Fault Localization."
[STVR'15] M. Papadakis, Y. Le Traon. "Metallaxis-FL: Mutation-Based Fault Localization."

35 mutants: change each comparison operator, and each assigned variable.
Run all 6 tests on each and count the tests that **flip**:

statement	mutants	fail $\rightarrow$ pass	pass $\rightarrow$ fail	$\mu(s)$
line 5 m = y	2	1	1	0.48
line 8	7	0	0	0.00
all the others	26	0	30	-0.02 to -0.08

- The only fail $\rightarrow$ pass: **m = y $\rightarrow$ m = x** on line 5. That mutant **is the fix**.
- A clearer gap than Tarantula (0.48 vs. 0, not 0.83 vs. 0.71). But it costs 35 mutants $\times$ 6 tests = **210** runs. A spectrum needs **6**.

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
3. Spectrum-based Fault Localization (SBFL)
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize
 Fault Localization in Practice

No formula wins everywhere. So give **many** signals to a ranker **learned** from known faults:

Approach	Features
Multric ²⁰	25 formulae, with learned weights
Savant ²¹	35 formulae + invariants that differ between passing and failing runs
FLUCCS ²²	formulae + code age, changes, size

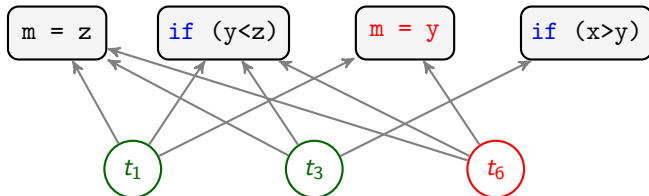
Fault localization became a **learning** problem.

²⁰[ICSME'14] J. Xuan, M. Monperrus. "Learning to Combine Multiple Ranking Metrics for Fault Localization."

²¹[ISSTA'16] T.-D. B. Le, D. Lo, C. Le Goues, L. Grunske. "A Learning-to-Rank Based Fault Localization Approach Using Likely Invariants."

²²[ISSTA'17] J. Sohn, S. Yoo. "FLUCCS: Using Code and Change Metrics to Improve Fault Localization."

Why pick formulae by hand? **GRACE** gives the whole coverage to a graph neural network.²³



- Nodes: statements and tests. Edges: coverage. Nothing is squeezed into four numbers.
- Defects4J, top-1: **195** bugs, vs. **166** for the best older method.

²³[ESEC/FSE'21] Y. Lou, Q. Zhu, J. Dong, X. Li, Z. Sun, D. Hao, L. Zhang, L. Zhang. "Boosting Coverage-Based Fault Localization via Graph-Based Representation Learning."

AutoFL: show an LLM the failing test. It calls **tools** to read the code, then names the buggy method and says **why**. Ask **several times**: how often it names a method is its score.²⁴

```
Test: mid(2, 1, 3) expected 2, got 1
LLM > get_covered_methods() -> [mid]
LLM > get_code_snippet(mid) -> "int mid(int x, ..."
LLM : "1 < 2 < 3, so the answer is x = 2.
      That case reaches line 5, which returns y."
Culprit: mid
```

²⁴[FSE'24] S. Kang, G. An, S. Yoo. "A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization."

- **Defects4J**: hundreds of real Java bugs, each with a failing test and the real fix.²⁵
- **acc@n**: the number of bugs whose faulty element is in the top n .
- **No family wins alone.** SBFL, MBFL, slicing, and IRFL find different bugs. Together they beat each one.²⁶ DeepFL learns how to combine them.²⁷

²⁵[ISSTA'14] R. Just, D. Jalali, M. D. Ernst. "Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs."

²⁶[TSE'21] D. Zou, J. Liang, Y. Xiong, M. D. Ernst, L. Zhang. "An Empirical Study of Fault Localization Families and Their Combinations."

²⁷[ISSTA'19] X. Li, W. Li, Y. Zhang, L. Zhang. "DeepFL: Integrating Multiple Fault Diagnosis Dimensions for Deep Fault Localization."

- Developers given a ranking do **not** go down the list. If the fault is not near the top, it does not help.²⁸
- What they want: the fault in the **top 5**, at **method** level, with a **reason**, and fast.²⁹
- The next lecture has a reader that will read **hundreds**: an **automatic repair** tool.

²⁸[ISSTA'11] C. Parnin, A. Orso. "Are Automated Debugging Techniques Actually Helping Programmers?"

²⁹[ISSTA'16] P. S. Kochhar, X. Xia, D. Lo, S. Li. "Practitioners' Expectations on Automated Fault Localization."

1. Delta Debugging (DD)
2. Information Retrieval based Fault Localization (IRFL)
3. Spectrum-based Fault Localization (SBFL)
4. Mutation-based Fault Localization (MBFL)
5. Learning to Localize

- Given a **faulty program**, an **oracle** (the correct program, callable but not readable), and one **failing input**, rank the statements by how likely each is to contain the fault.
- Graded mostly on **hidden** programs: they are larger and harder than the public examples, so a perfect score on the examples does not mean a perfect grade.
- Please see this document on GitHub:

<https://github.com/ku-plrg-classroom/js-fl>

- The due date is **23:59 on October 26 (Mon.)**, and please submit it to [LMS](#).

- Automated Program Repair

Jihyeok Park

`jihyeok_park@korea.ac.kr`

<https://plrg.korea.ac.kr>